

Assignment 3 - Advanced Programming 2026

Anton Bang Lejbølle & Jonas Dennis Hansen

September 2026

1 Introduction

For this assignment, we do not have any general concerns or have found any ambiguities in the problem text. We are confident that our solution is of high quality as it is functional and passes all the test cases we have written for it. There might still be some edge cases that we have not considered, but we believe that our solution is robust and handles the majority of scenarios effectively.

Our parser is functional, and we have written various test cases to cover its functionality. The test cases ensure that function application is parsed correctly and with the correct associativity, and that the equality and power operators have the correct precedence and associativity. We have also tested the print, put, and get operations, including string literals and keywords, as well as lambdas, let-bindings, loops, and try-catch expressions. We have also written test cases for invalid expressions to ensure that the parser correctly rejects inputs that are not part of the language.

We have also tested the different precedence levels in the grammar to ensure that expressions are parsed into the expected AST. Since the assignment required us to resolve operator precedence ambiguities, we have written tests that cover expressions involving multiple operators and function applications. Overall, we believe that the parser correctly implements the functionality required by the assignment.

2 Answers to questions

1. Show the final and complete grammar with left-recursion removed and all ambiguities resolved.

The final and complete grammar with left-recursion removed and all ambiguities resolved is:

Listing 1: The final and complete grammar

```

Atom ::= var
      | int
      | bool
      | "(" Exp ")"

FExp' ::= (* empty *)
      | Atom FExp'

FExp ::= Atom FExp'

LExp ::= FExp
      | "if" Exp "then" Exp "else" Exp
      | "\" var "->" Exp
      | "try" Exp "catch" Exp
      | "let" var "=" Exp "in" Exp
      | "loop" var "=" Exp "for" var "<" Exp "do" Exp

Exp4 ::= LExp
      | "print" string Atom
      | "get" Atom
      | "put" Atom Atom

Exp3' ::= (* empty *)
      | "***" Exp3

Exp3 ::= Exp4 Exp3'

Exp2' ::= (* empty *)
      | "*" Exp3 Exp2'
      | "/" Exp3 Exp2'

Exp2 ::= Exp3 Exp2'

Exp1' ::= (* empty *)
      | "+" Exp2 Exp1'
      | "-" Exp2 Exp1'

Exp1 ::= Exp2 Exp1'

Exp0' ::= (* empty *)
      | "==" Exp1 Exp0'

Exp0 ::= Exp1 Exp0'

Exp ::= Exp0

```

2. Why might or might it not be problematic for your implementation that the grammar has operators * and ** where one is a prefix of the other?

It is not problematic for our implementation that the grammar has operators * and ** where one is a prefix of the other, because the ** operator has the highest precedence of all operators, which means that only if it is not the ** operator then it is checked whether it is the * operator. Additionally the * operator has to have something other than * afterwards for it to be the * operator. Thereby there is no doubt that if there are ** it is the ** operator and not 2 * operators.

3. Where in your parser is backtracking (try) strictly necessary, and where is it merely convenient? For one try you could remove, state what changes: the language recognised, the error messages, or the running time?

In IVName and in IKeyword backtracking (try) is strictly necessary. In IVName it is strictly necessary, because only after the variable name has been parsed, then the parser can check if the variable name is already a keyword, and thereby should not have been parsed as a variable name. If try is removed in IVName then it will successfully parse all keywords as variable names, but then the parser will fail with "Unexpected keyword" since what has been parsed is in the keywords list.

In IKeyword it is strictly necessary, because only after the keyword has been parsed, then the parser can check if the keyword is followed by something that is AlphaNum, and thereby should not have been parsed as the keyword. This will become a problem when parsing variable names that start with "true" or "false". If try is removed in IKeyword then as an example, "true" will parse "true" as "CstBool True", but then the parser will fail since it is followed by something that is AlphaNum. This could be avoided by changing the order of the choices in the atom parser, such that variables are parsed before bools, but if we want the order to not matter, then backtracking is strictly necessary.

4. Give an example of where in your parser it matters that choice is ordered (picks the first successful parse) (or equivalently for < | >, if you have used that instead of choice). If no such case occurs, explain why.

In all the chains that end with pure x it matters, because pure x will always successfully parse, so this should only happen if the previous choices did not successfully parse. As an example in pExp0, if the pure x choice is first then it will never pick the choice of parsing the == operator.

In the few other cases it does not matter, because each choice will only successfully parse when the string has a specific start, and because there is no overlap between the specifics starts, then the order does not matter. In the

atom parser, the choices will only successfully parse on a specific set of strings, but since there is not overlap between the sets that will successfully parse for each choice, then the order does not matter.

5. Suppose APL is extended with comments notation (such as line comments or block comments as in Haskell). How would you implement this in your parser, and would it require many modifications?

We would add something that starts the comment, and then something that parses everything up until the indicator for ending the comment. We would then include it in the lexeme function that removes whitespaces, so that it would also consume comments. Then the function could consume all the characters that do not affect the computations of the program. It would not require many modifications as the rest of the parser can still work while remaining unchanged.